

Informe Técnico: Middleware Distribuido de Recursos Concurrentes

Agustín Messana Gullielmi Nahuel Deppen Felipe Pineda
Gaspar Emilio Rubies

Resumen

Este informe detalla el diseño, implementación y validación de un middleware distribuido e interoperable para la asignación concurrente de recursos. La solución integra un planificador global en Erlang y agentes locales en C, garantizando una comunicación eficiente no bloqueante mediante `epoll`, descubrimiento dinámico por UDP broadcast y un mecanismo robusto basado en Orden Total para la prevención estática de bloqueos mutuos (*deadlocks*).

1. Diseño del Sistema e Interoperabilidad

El middleware adopta un enfoque híbrido combinando dos capas complementarias:

- **Capa de Coordinación Global (Erlang):** Implementa el planificador central (`scheduler.erl`) bajo el modelo de actores. Centraliza la topología del clúster y coordina las demandas concurrentes.
- **Capa de Agente Local (Lenguaje C):** Ejecuta una arquitectura impulsada por eventos uniproceto mediante `epoll` de Linux, gestionando el hardware local (`ResourceManager`) sin sobrecostos de hilos tradicionales.

1.1. Topología y Protocolo de Interoperabilidad (Erlang ↔ C)

La infraestructura se estructura de forma parcialmente mallada y dinámica. Los agentes locales se descubren asíncronamente en red mediante sockets UDP emitiendo un aviso de actividad `ANNOUNCE` cada 5 segundos al puerto 8888. Cada agente mantiene una tabla interna de nodos en memoria, purgando pasivamente a aquellos vecinos que registren más de 15 segundos de inactividad.

La interoperabilidad entre componentes heterogéneos se abstrae mediante sockets TCP sobre interfaces de red activas. El planificador solicita la topología enviando textualmente `GET NODES\N`. El agente en C responde volcando la lista formateada con delimitadores estrictos: caracteres de dos puntos (`:`) para atributos de un mismo nodo y puntos y comas (`;`) para separar registros de vecinos independientes, culminando con un salto de línea final. Esto permite que el canal de control orqueste tanto el entorno local como el ruteo transaccional de peticiones remotas distribuidas en hosts físicos independientes utilizando las direcciones IP extraídas dinámicamente por la capa de descubrimiento.

1.2. Diagrama de Secuencia del Flujo Global

El flujo temporal se divide en dos fases: (1) *Descubrimiento local*, donde el planificador obtiene síncronamente la topología de red, y (2) *Petición distribuida*, donde las demandas compuestas son linealizadas por Erlang e inyectadas al agente local, el cual procesa sus recursos nativos y rutea de forma transparente las solicitudes remotas mediante conexiones TCP síncronas hacia las interfaces públicas de los agentes vecinos.

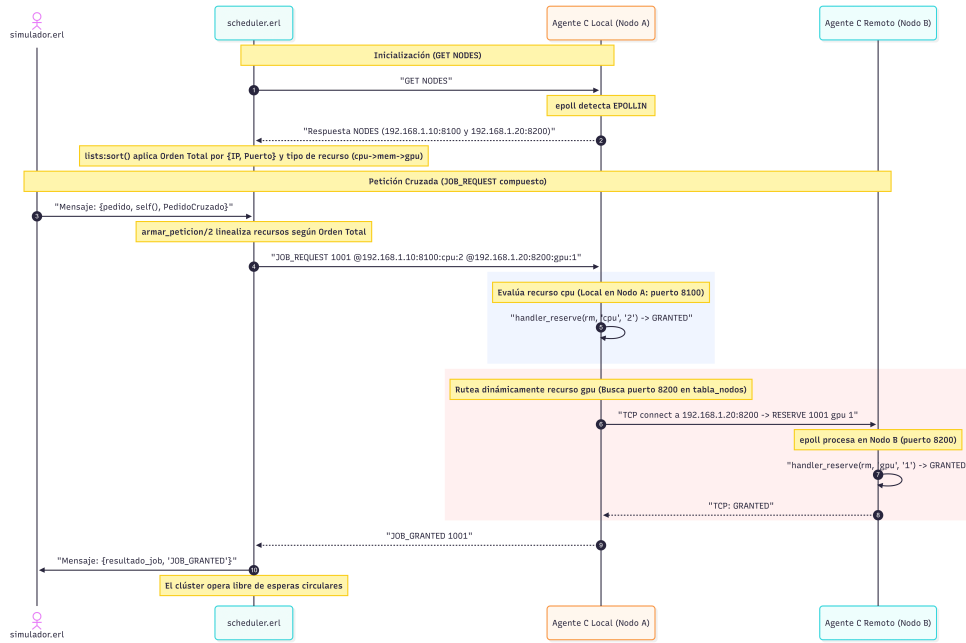


Figura 1: Diagrama de secuencia de comunicación local (Erlang ↔ C) y distribuida entre agentes remotos.

2. Problemas Encontrados y Soluciones Técnicas

Durante la fase de integración de las capas de transporte y lógica de negocio, se detectaron y resolvieron tres problemáticas críticas de sistemas distribuidos:

1. **Fragmentación de Paquetes en la Red TCP:** Inicialmente, la capa de transporte en Erlang operaba con sockets planos (`{packet, 0}`). Debido a la naturaleza orientada a flujos de TCP, las respuestas de C llegaban fragmentadas o aglutinadas, corrompiendo el parseo. **Solución:** Se configuraron los descriptores en `cliente_tcp.erl` con la bandera `{packet, line}`, delegando el reensamblado en la máquina virtual BEAM, la cual despacha mensajes unívocos al buzón únicamente tras identificar el delimitador `\n`.
2. **Condición de Carrera en el Descubrimiento Dinámico:** En entornos locales concurrentes, el script de estrés lanzaba el planificador inmediatamente después de inicializar los agentes. Al no completarse las ráfagas UDP de presencia, `GET NODES` retornaba una lista incompleta, haciendo crashear a Erlang por fallos de emparejamiento (*badmatch*). **Solución:** Se incorporó un retardo preventivo de hasta 4 segundos mediante `sleep` en el script de pruebas (`test_deadlock.sh`), otorgando el margen físico necesario para que el lazo de eventos en C complete sus 2 segundos obligatorios de arranque pasivo y establezca la topología UDP antes de habilitar las conexiones de control de Erlang.
3. **Advertencias de Truncamiento y Signo en el Compilador:** Al compilar bajo las banderas estrictas del Makefile (`-Wall -Wextra`), gcc advertía riesgos de truncamiento por el uso de `strncpy` en buffers de copia para rollbacks, y colisiones de signo al comparar enteros de asignación (`unsigned int` contra `int` en `handler_release`). **Solución:** Se reemplazó el copiado primitivo por la función segura `snprintf` e incorporaron conversiones explícitas (*casts*) de tipo en las estructuras de decisión del administrador de recursos.

3. Estrategia de Prevención de Deadlocks

Para evitar la posibilidad de interbloqueos mutuos en el clúster sin incurrir en algoritmos dinámicos de detección de ciclos, el diseño implementa una estrategia de **prevención estática**

mediante el establecimiento de un **Orden Total** en la adquisición de hardware distribuido, bajo el modelo de recursos actual, evitando la condición de espera circular de Coffman.

3.1. Algoritmo de Ordenamiento Jerárquico Global

La linealización se ejecuta centralizadamente en Erlang dentro de `scheduler:armar_peticion/2`. Antes de inyectar el comando `JOB_REQUEST` al socket, los requerimientos se ordenan estrictamente bajo dos niveles jerárquicos para garantizar un Orden Total unívoco:

1. **Nivel de Red (IP y Puerto):** Las peticiones se ordenan utilizando la tupla jerárquica del par `{IP, Puerto}` del nodo destino. Esto resulta indispensable para romper la ambigüedad y discriminar nodos de forma unívoca cuando múltiples instancias de simulación concurren dentro de una misma interfaz de red local (*localhost*).
2. **Nivel de Recurso (Prioridad Estática):** Si múltiples demandas convergen sobre un mismo nodo físico, Erlang desempata aplicando pesos discretos fijos mediante la función de precedencia `prioridad_recurso/1`: `cpu` (1) $\rightarrow$ `mem` (2) $\rightarrow$ `gpu` (3). Este esquema se encuentra completamente acoplado con la estructura numérica interna implementada en el agente de C (0 $\rightarrow$ `cpu`, 1 $\rightarrow$ `mem`, 2 $\rightarrow$ `gpu`), garantizando consistencia entre la representación utilizada por Erlang y la implementada en el agente C.

```

1  armar_peticion(IdJob, RecursosPedidos) ->
2      PeticionesOrdenadas = lists:sort(
3          fun({IpA, PuertoA, RecursoA, _}, {IpB, PuertoB, RecursoB, _}) ->
4              case {IpA, PuertoA} == {IpB, PuertoB} of
5                  true -> prioridad_recurso(RecursoA) =< prioridad_recurso(
6                      RecursoB);
7                  false -> {IpA, PuertoA} =< {IpB, PuertoB}
8              end
9          end,
10         RecursosPedidos
11     ),
12     ComandoBase = io_lib:format("JOB_REQUEST_~p", [IdJob]),
13     Fragmentos = [
14         io_lib:format("_@~s:~s:~s~p", [IP, Puerto, Recurso, Cantidad])
15         || {IP, Puerto, Recurso, Cantidad} <- PeticionesOrdenadas
16     ],
17     lists:flatten([ComandoBase | Fragmentos]).

```

Listing 1: Mecanismo de linealización y orden total real en scheduler.erl

3.2. Ejemplo Paso a Paso con Dos Nodos

Para demostrar cómo se evita la espera circular, considérese el escenario crítico evaluado en los ensayos empíricos: dos agentes concurrentes compartiendo la misma interfaz de red (172.24.129.190). El **Nodo A** opera en el puerto 8100 y posee cores de `cpu`, mientras que el **Nodo B** opera en el puerto 8200 y cuenta con aceleradores `gpu`. Dos procesos del simulador disparan demandas simultáneas cruzadas:

- **Job 1** solicita de forma nativa: `[{172.24.129.190, 8100, "cpu", 2}, {172.24.129.190, 8200, "gpu", 1}]`
- **Job 2** solicita de forma nativa: `[{172.24.129.190, 8200, "gpu", 1}, {172.24.129.190, 8100, "cpu", 2}]`

Ejecución bajo el algoritmo del Middleware:

1. **Fase de Linealización:** Al procesar las solicitudes concurrentes, el planificador aplica la función `armar_peticion/2`. El algoritmo evalúa el primer nivel jerárquico por la

tupla {IP, Puerto}. Como el puerto 8100 es numéricamente menor que el 8200, ambas listas de requerimientos son forzadas a alinearse bajo la misma secuencia exacta. El string final transmitido al agente C adopta estrictamente la forma: "JOB_REQUEST 1001 @172.24.129.190:8100:cpu:2 @172.24.129.190:8200:gpu:1".

2. **Fase de Competencia Segura:** Ambos Jobs son obligados a competir secuencialmente por el primer eslabón del Orden Total establecido: la *cpu* del puerto 8100 (Nodo A).
3. **Resolución de Contención:** Si el *Job 1* intercepta primero el descriptor local, el *ResourceManager* nativo en C del Nodo A ejecuta `handler_reserve` y le concede la reserva (*GRANTED*), habilitando al agente a conectarse por TCP al puerto 8200 para pedir la *gpu* remota. Cuando el *Job 2* intenta procesar su cadena, queda retenido en el eslabón inicial de la *cpu*. Al estar agotada, el agente C del Nodo A retorna inmediatamente un veredicto de escasez (*JOB_DENIED*).
4. **Ausencia de Bloqueo:** El *Job 2* retrocede de forma limpia mediante las rutinas de rollback, liberando cualquier intento parcial y entrando en un ciclo de reintento pasivo temporizado. El *Job 1* completa su circuito de manera exitosa sin quedar retenido de forma cruzada, erradicando el interbloqueo.

4. Validación Empírica y Evidencia de Ejecución

Para verificar la robustez de la solución frente a condiciones de estrés, se ejecutó el script automatizado `test_deadlock.sh`. El sistema inicializó con éxito múltiples instancias concurrentes del agente en C y acopló el planificador global en Erlang.

A continuación, se detalla el extracto real extraído del archivo de traza `scheduler.log` que demuestra el comportamiento del middleware frente a la contención del escenario crítico del §6:

```

1  NUEVO JOB 2: Solicitado por Pid <0.83.0> - JOB_REQUEST 2 @172.24.129.190:8100:cpu:2 @172
   .24.129.190:8200:gpu:1
2  NUEVO JOB 3: Solicitado por Pid <0.84.0> - JOB_REQUEST 3 @172.24.129.190:8100:cpu:2 @172
   .24.129.190:8200:gpu:1
3  RESOLUCION JOB 2: JOB_GRANTED
4  REINTENTO JOB 3 (intento 2/3) tras JOB_DENIED
5  REINTENTANDO JOB 3 (intento 2): JOB_REQUEST 3 @172.24.129.190:8100:cpu:2 @172
   .24.129.190:8200:gpu:1
6  REINTENTO JOB 3 (intento 3/3) tras JOB_DENIED
7  REINTENTANDO JOB 3 (intento 3): JOB_REQUEST 3 @172.24.129.190:8100:cpu:2 @172
   .24.129.190:8200:gpu:1
8  JOB 3 DESCARTADO tras 3 intentos: JOB_DENIED

```

Listing 2: Extracto de `scheduler.log` durante la ejecución del test de deadlock distribuido

Como se observa en la traza empírica, gracias a la linealización impuesta por el Orden Total, ambos trabajos compitieron de manera ordenada por el primer eslabón jerárquico (la *cpu* del puerto 8100). El *Job 2* interceptó primero el recurso obteniendo el veredicto de *JOB_GRANTED*.

Por su parte, las solicitudes concurrentes subsiguientes (identificadas en las trazas de control del planificador y los agentes bajo las iteraciones del *Job 3* y sus reintentos de red) retrocedieron limpiamente tras recibir *JOB_DENIED*, liberando cualquier intento parcial mediante las rutinas de rollback y ejecutando su ciclo de reintento pasivo temporizado. Al mantenerse la alta contención durante los ciclos posteriores, el trabajo terminó siendo descartado tras agotar sus capacidades reguladas de reintento. Este comportamiento evidencia que, en el escenario evaluado, no se produjeron bloqueos mutuos ni situaciones de espera circular, validando la solidez macro del algoritmo distribuido.

5. Roles del Equipo y Conclusiones

La arquitectura implementada valida la solidez de combinar Erlang para la coordinación de alto nivel y C para la multiplexación eficiente de E/S nativa a bajo nivel.

5.1. División de Roles y Contribuciones Individuales

Para garantizar un desarrollo eficiente y modular, las responsabilidades del proyecto se distribuyeron de la siguiente manera:

- **Agustín Messana Gullielmi (Capa de Orquestación en Erlang):** Se encargó del desarrollo del planificador central (`scheduler.erl`). Diseñó el bucle de control, la recepción de eventos del clúster y el algoritmo de ordenamiento por prioridad de recursos utilizado para prevenir los interbloqueos.
- **Nahuel Deppen (Infraestructura del Servidor en C):** Responsable de la arquitectura base del servidor nativo en C. Implementó el manejo de conexiones mediante la API `epoll` de Linux, configuró los sockets no bloqueantes y programó las rutinas para aceptar y rutear las peticiones por TCP.
- **Felipe Pineda (Lógica del Administrador de Recursos):** Diseñó e implementó el componente `ResourceManager` en C. Se encargó del manejo de las estructuras de datos en memoria, las colas de espera del hardware y las funciones de validación, incremento y decremento de recursos.
- **Gaspar Emilio Rubies (Descubrimiento de Red e Integración):** Desarrolló el sistema de descubrimiento automático de nodos mediante sockets UDP broadcast. Diseñó el aviso de actividad, la extracción dinámica de datos de red de los vecinos y unificó el código para la integración final entre Erlang y C.

5.2. Conclusiones y Aprendizajes del Proyecto

El desarrollo de este middleware permitió dimensionar los desafíos reales de la programación distribuida, tales como la gestión de buffers ante la fragmentación de flujos de bytes sobre TCP y el control de condiciones de carrera en protocolos no orientados a la conexión como UDP.

Asimismo, el proyecto demostró que la aplicación de políticas de prevención estática mediante Orden Total constituye una alternativa eficiente y de bajo costo computacional frente a algoritmos de detección dinámica de ciclos para el escenario abordado.